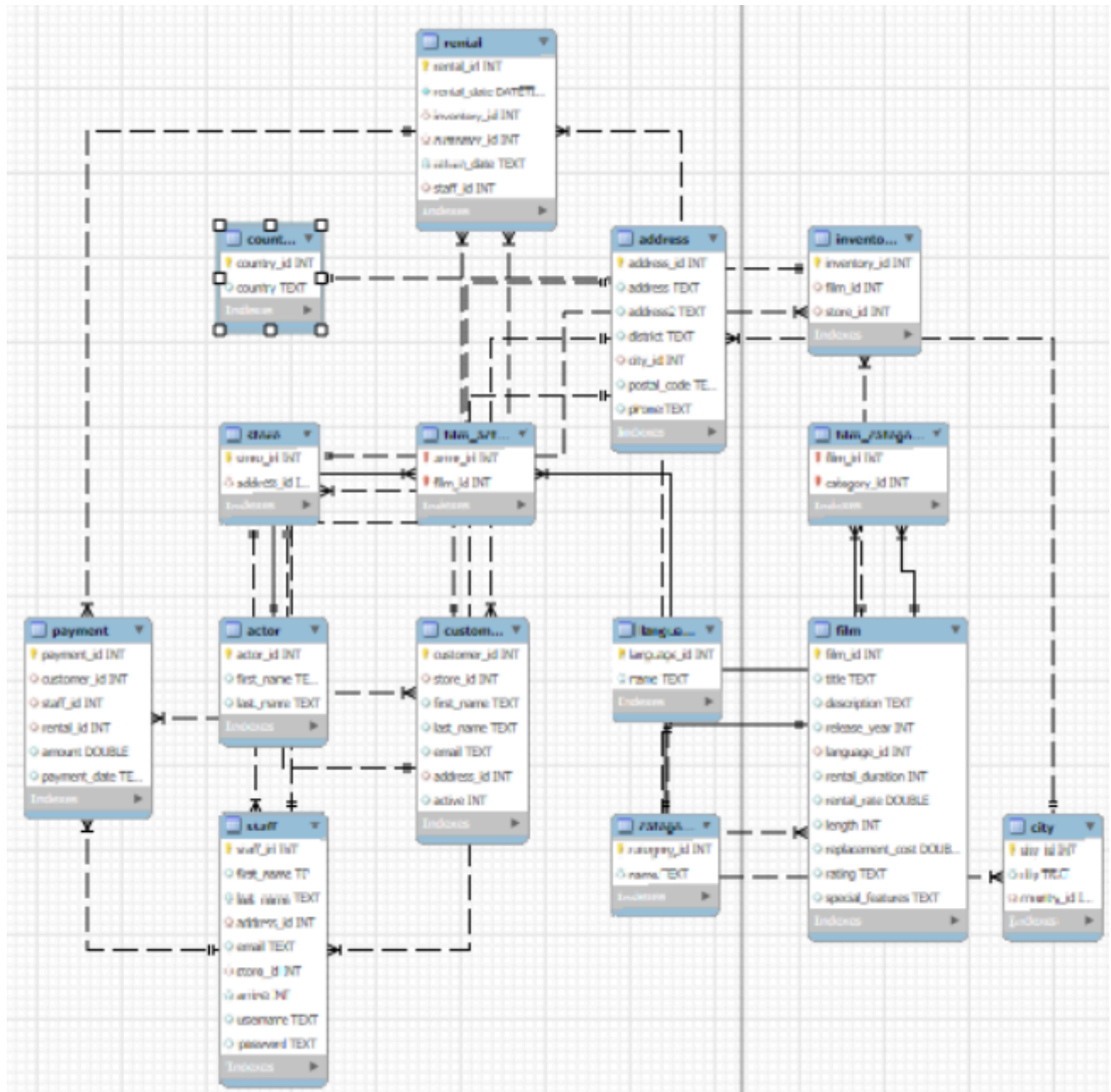


DB Assignment 4
Emily Foreacre
11/4/2025



declaring the primary key, foreign key, and unique constraint for each table

actor table

alter table actor

add primary key (actor_id);

address table

alter table address

add primary key (address_id);

alter table address

add constraint address_city_id foreign key (city_id) references city(city_id);

category table

alter table category

add primary key (category_id);

city table

alter table city

add primary key (city_id);

alter table city

add constraint city_country_id foreign key (country_id) references country(country_id);

country table

alter table country

add primary key (country_id);

customer table

alter table customer

add primary key (customer_id);

alter table customer

add constraint customer_store_id foreign key (store_id) references store(store_id);

alter table customer

add constraint customer_address_id foreign key (address_id) references
address(address_id);

film table

alter table film

add primary key (film_id);

alter table film

add constraint film_language_id foreign key (language_id) references
language(language_id);

film_actor table

```
alter table film_actor
    add primary key (actor_id, film_id);
alter table film_actor
    add constraint film_actor_actor_id foreign key (actor_id) references actor(actor_id),
    add constraint film_actor_film_id foreign key (film_id) references film(film_id);
```

rental table

```
alter table rental
    add primary key (rental_id);
alter table rental
    modify rental_date datetime not null;
alter table rental
    add constraint rental_inventory_customer unique (rental_date,inventory_id,
customer_id);
alter table rental
    add constraint rental_inventory_id foreign key (inventory_id) references
inventory(inventory_id),
    add constraint rental_customer_id foreign key (customer_id) references
customer(customer_id),
    add constraint rental_staff_id foreign key (staff_id) references staff(staff_id);
```

staff table

```
alter table staff
    add primary key (staff_id);
alter table staff
    add constraint staff_address_id foreign key (address_id) references
address(address_id),
    add constraint staff_store_id foreign key (store_id) references store(store_id);
```

store table

```
alter table store
    add primary key (store_id);
alter table store
```

```
    add constraint store_address_id foreign key (address_id) references  
address(address_id);
```

film_category table

```
alter table film_category
```

```
    add primary key (film_id, category_id);
```

```
alter table film_category
```

```
    add constraint film_category_film_id foreign key (film_id) references film(film_id),
```

```
    add constraint film_category_category_id foreign key (category_id) references  
category(category_id);
```

inventory table

```
alter table inventory
```

```
    add primary key (inventory_id);
```

```
alter table inventory
```

```
    add constraint inventory_film_id foreign key (film_id) references film(film_id),
```

```
    add constraint inventory_store_id foreign key (store_id) references store(store_id);
```

language table

```
alter table language
```

```
    add primary key (language_id);
```

payment table

```
alter table payment
```

```
    add primary key (payment_id);
```

```
alter table payment
```

```
    add constraint payment_customer_id foreign key (customer_id) references  
customer(customer_id),
```

```
    add constraint payment_staff_id foreign key (staff_id) references staff(staff_id),
```

```
    add constraint payment_rental_id foreign key (rental_id) references rental(rental_id);
```

1. What is the average length of films in each category? List the results in alphabetic order of categories.

```
select c.name as category, avg(f.length) as avg_length -- calculate average length of films
from film f
join film_category fc ON f.film_id = fc.film_id -- join films to category
join category c ON fc.category_id = c.category_id
group by c.name
order by c.name; -- sort alphabetically
```

	category	avg_length
▶	Action	111.6094
	Animation	111.0152
	Children	109.8000
	Classics	111.6667
	Comedy	115.8276
	Documentary	108.7500
	Drama	120.8387
	Family	114.7826
	Foreign	121.6986
	Games	127.8361
	Horror	112.4821
	Music	113.6471
	New	111.1270
	Sci-Fi	108.1967

The query tells you how long films typically are for each category, and lists the categories in alphabetical order.

2. Which categories have the longest and shortest average film lengths?

```
with avg_length as (
  select c.name, avg(f.length) as avg_length -- calculate average length of films
  from film f
  join film_category fc on f.film_id = fc.film_id -- join films to category
  join category c on fc.category_id = c.category_id
  group by c.name
)
select 'longest' as which, name, (avg_length) as avg_length
```

```

from avg_length
where avg_length = (select MAX(avg_length) from avg_length) -- calculate maximum avg length
union all
select 'shortest' as which, name, (avg_length) as avg_length
from avg_length
where avg_length = (select MIN(avg_length) from avg_length) -- calculate minimum avg length
order by which, name;

```

	which	name	avg_length
▶	longest	Sports	128.2027
	shortest	Sci-Fi	108.1967

This query tells you which category has the longest average films and which has the shortest, it also shows you the average lengths of each category.

3. Which customers have rented action but not comedy or classic movies?

```

select c.customer_id, c.first_name, c.last_name, c.email
from customer c
join rental r on c.customer_id = r.customer_id -- join customer to rental
join inventory i on r.inventory_id = i.inventory_id -- join rental to inventory
join film f on i.film_id = f.film_id -- join inventory to film
join film_category fc on f.film_id = fc.film_id -- join film to film_category
join category cat on fc.category_id = cat.category_id -- join film_category to category
group by c.customer_id, c.first_name, c.last_name, c.email
having sum(cat.name = 'Action') > 0 and sum(cat.name in ('Comedy','Classics')) = 0 -- must
have rented as least 1 action film and 0 comedy and classic films
order by c.first_name, c.last_name; -- sort alphabetically

```

	customer_id	first_name	last_name	email
▶	139	AMBER	DIXON	AMBER.DIXON@sakilacustomer.org
	232	CONSTANCE	REID	CONSTANCE.REID@sakilacustomer.org
	171	DOLORES	WAGNER	DOLORES.WAGNER@sakilacustomer.org
	433	DON	BONE	DON.BONE@sakilacustomer.org
	17	DONNA	THOMPSON	DONNA.THOMPSON@sakilacustomer.org
	432	EDWIN	BURK	EDWIN.BURK@sakilacustomer.org
	213	GINA	WILLIAMSON	GINA.WILLIAMSON@sakilacustomer.org
	250	JO	FOWLER	JO.FOWLER@sakilacustomer.org
	164	JOANN	GARDNER	JOANN.GARDNER@sakilacustomer.org
	350	JUAN	FRALEY	JUAN.FRALEY@sakilacustomer.org
	361	LAWRENCE	LAWTON	LAWRENCE.LAWTON@sakilacustomer.org
	323	MATTHEW	MAHAN	MATTHEW.MAHAN@sakilacustomer.org
	223	MELINDA	FERNANDEZ	MELINDA.FERNANDEZ@sakilacustomer....
	445	MICHEAL	FORMAN	MICHEAL.FORMAN@sakilacustomer.org

This lists all customers who have rented at least 1 action film, without ever renting Comedy or Classics movies.

4. Which actor has appeared in the most English-language movies?

with actor_counts as (

```
select a.actor_id, a.first_name, a.last_name, count(distinct fa.film_id) as english_film_count --
count the distinct film actor film id
```

```
from actor a
```

```
join film_actor fa on a.actor_id = fa.actor_id -- join actor to film_actor
```

```
join film f on fa.film_id = f.film_id -- join film_actor to film
```

```
join language l on f.language_id = l.language_id -- join film to language
```

```
where l.name = 'English' -- the only films it will be counting are those in English
```

```
group by a.actor_id, a.first_name, a.last_name
```

```
)
```

```
select actor_id, first_name, last_name, english_film_count
```

```
from actor_counts
```

```
where english_film_count = (
```

```
select max(english_film_count) from actor_counts -- select the actor with the most english film
```


)

order by first_name, last_name;

	actor_id	first_name	last_name	english_film_count
▶	107	GINA	DEGENERES	42

This query shows you the actor who has appeared in the most English speaking films. Their actor id, and their first and last name; along with the number of English films they've been in.

5. How many distinct movies were rented for exactly 10 days from the store where Mike works?

```
select count(distinct i.film_id) as distinct_movies_rented_10_days
```

```
from rental r
```

```
join inventory i on r.inventory_id = i.inventory_id -- join rental to inventory
```

```
join staff s on s.store_id = i.store_id -- join store to staff
```

```
where datediff(r.return_date, r.rental_date) = 10 and i.store_id in ( -- the 10 day difference of the  
date from rental_date to return_date
```

```
    select distinct store_id
```

```
    from staff
```

```
    where first_name = 'Mike' -- only count the store where Mike works
```

```
);
```

	distinct_movies_rented_10_days
▶	61

This shows the amount of movies that were rented in a 10 day time span at the store Mike works.

6. Alphabetically list actors who appeared in the movie with the largest cast of actors.

```
select a.first_name, a.last_name, cast_count.num_actors as cast_size
from actor a
join film_actor fa on a.actor_id = fa.actor_id -- join actor to film_actor
join (
    select fa2.film_id, count(fa2.actor_id) as num_actors -- count film actors
    from film_actor fa2
    group by fa2.film_id
    order by num_actors desc
    limit 1 -- limit the film to one
) as cast_count on fa.film_id = cast_count.film_id
order by a.first_name, a.last_name; -- sort alphabetically
```

	first_name	last_name	cast_size
▶	BURT	POSEY	15
	CAMERON	ZELLWEGER	15
	CHRISTIAN	NEESON	15
	FAY	WINSLET	15
	JAYNE	NOLTE	15
	JULIA	BARRYMORE	15
	JULIA	ZELLWEGER	15
	LUCILLE	DEE	15
	MENA	HOPPER	15
	MENA	TEMPLE	15
	REESE	KILMER	15
	SCARLETT	DAMON	15
	VAL	BOLGER	15
	WALTER	TORN	15
	WOODY	HOFFMAN	15

This lists the actors, first and last name, that appeared in the movie with the largest number of actors in alphabetical order. Along with the amount of actors in the cast.